

DEPARTMENT OF CSE(AI&ML)
AL3501-NATURAL LANGUAGE PROCESSING

UNIT I

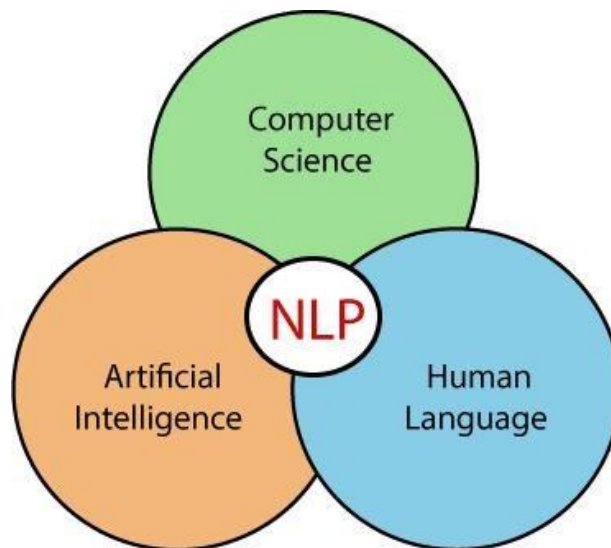
INTRODUCTION

Origins and challenges of NLP – Language Modeling: Grammar-based LM, Statistical LM - Regular Expressions, Finite-State Automata – English Morphology, Transducers for lexicon and rules, Tokenization, Detecting and Correcting Spelling Errors, Minimum Edit Distance

Natural Language Processing

Language is a method of communication with the help of which we can speak, read and write. Natural Language Processing (NLP) is a subfield of Computer Science that deals with Artificial Intelligence (AI), which enables computers to understand and process human language.

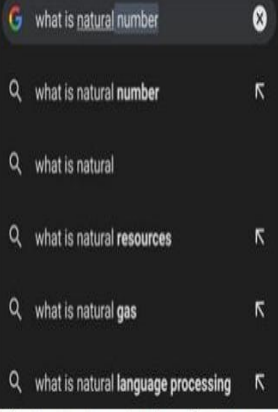
We can say that NLP is a part of **Computer Science**, **Human language**, and **Artificial Intelligence**. It is the technology that is used by machines to understand, analyse, manipulate, and interpret human's languages. It helps developers to organize knowledge for performing tasks such as **translation, automatic summarization, Named Entity Recognition (NER), speech recognition, relationship extraction, and topic segmentation**.



Technically, the main task of NLP would be to program computers for analyzing and processing huge amount of natural language data.

Are You Already Using Natural Language Processing?

How many of you believe you've interacted with an application powered by Natural Language Processing (NLP) today or in the past week? Take a moment to think about it. Now, let's uncover some everyday interactions you might have had:

1. Zomato Interactive Assistant: Have you ever ordered food online and chatted with a customer support bot?  This is Zomato's chatbot. It leverages NLP to understand your queries and respond in a human-like manner. These bots can assist with order status, address concerns, or even recommend dishes!	2. Voice Search & Assistants: When you say, "Hey Google" or "Siri", and ask a question or give a command, you're using NLP.  Voice-activated assistants rely heavily on NLP to understand and process our spoken language. It's not just about recognizing words, but understanding intent.	3. Autocomplete & Predictive Text: Have you ever started typing a message or search, and your device suggests the next word?  This is NLP in action! Predictive text models anticipate what you're likely to type next based on your history and common language patterns.	4. Translation Tools: Have you used tools like Google Translate to decipher a phrase in a different language or to communicate with someone who speaks another language?  NLP powers these tools, enabling real-time translations and helping bridge language barriers.
---	---	---	---

History of NLP

We have divided the history of NLP into four phases. The phases have distinctive concerns and styles.

First Phase (Machine Translation Phase) - Late 1940s to late 1960s

The work done in this phase focused mainly on machine translation (MT). This phase was a period of enthusiasm and optimism.

Let us now see all that the first phase had in it –

The research on NLP started in early 1950s after Booth & Richens' investigation and

Weaver's memorandum on machine translation in 1949.

1954 was the year when a limited experiment on automatic translation from Russian to English demonstrated in the Georgetown-IBM experiment.

In the same year, the publication of the journal MT (Machine Translation) started. The first international conference on Machine Translation (MT) was held in 1952 and second was held in 1956.

In 1961, the work presented in Teddington International Conference on Machine Translation of Languages and Applied Language analysis was the high point of this phase.

Second Phase (AI Influenced Phase) – Late 1960s to late 1970s

In this phase, the work done was majorly related to world knowledge and on its role in the construction and manipulation of meaning representations. That is why, this phase is also called AI-flavored phase.

The phase had in it, the following –

In early 1961, the work began on the problems of addressing and constructing data or knowledge base. This work was influenced by AI.

In the same year, a BASEBALL question-answering system was also developed. The input to this system was restricted and the language processing involved was a simple one. A much advanced system was described in Minsky (1968). This system, when compared to the BASEBALL question-answering system, was recognized and provided for the need of inference on the knowledge base in interpreting and responding to language input.

Third Phase (Grammatico-logical Phase) – Late 1970s to late 1980s

This phase can be described as the grammatico-logical phase. Due to the failure of practical system building in last phase, the researchers moved towards the use of logic for knowledge representation and reasoning in AI.

The third phase had the following in it –

The grammatico-logical approach, towards the end of decade, helped us with powerful general-purpose sentence processors like SRI's Core Language Engine and Discourse Representation Theory, which offered a means of tackling more extended discourse.

In this phase we got some practical resources & tools like parsers, e.g. AlveyNatural Language Tools along with more operational and commercial systems, e.g. for database query.

The work on lexicon in 1980s also pointed in the direction of grammatico- logical approach.

Fourth Phase (Lexical & Corpus Phase) – The 1990s

We can describe this as a lexical & corpus phase. The phase had a lexicalized approach to grammar that appeared in late 1980s and became an increasing influence. There was a revolution in natural language processing in this decade with the introduction of machine learning algorithms for language processing.

Study of Human Languages

Language is a crucial component for human lives and also the most fundamental aspect of our behavior. We can experience it in mainly two forms - written and spoken. In the written form, it is a way to pass our knowledge from one generation to the next. In the spoken form, it is the primary medium for human beings to coordinate with each other in their day-to-day behavior. Language is studied in various academic disciplines. Each discipline comes with its own set of problems and a set of solution to address those.

Consider the following table to understand this –

Discipline	Problems	Tools
Linguists	How phrases and sentences can be formed with words? What curbs the possible meaning for a sentence?	Intuitions about well-formedness and meaning. Mathematical model of structure. For example, model theoretic semantics, formal language theory.
Psycholinguists	How human beings can identify the structure of sentences? How the meaning of words can be identified? When does understanding take place?	Experimental techniques mainly for measuring the performance of human beings. Statistical analysis of observations.

Philosophers	How do words and sentences acquire the meaning? How the objects are identified by the words? What is meaning?	Natural language argumentation by using intuition. Mathematical models like logic and model theory.
Computational Linguists	How can we identify the structure of a sentence How knowledge and reasoning can be modeled? How we can use language to accomplish specific tasks?	Algorithms Data structures Formal models of representation and reasoning. AI techniques like search & representation methods.

CHALLENGES

Ambiguity and Uncertainty in Language

Ambiguity, generally used in natural language processing, can be referred as the ability of being understood in more than one way. In simple terms, we can say that ambiguity is the capability of being understood in more than one way. Natural language is very ambiguous. NLP has the following types of ambiguities –

Lexical Ambiguity

The ambiguity of a single word is called lexical ambiguity. For example, treating the word **silver** as a noun, an adjective, or a verb.

Syntactic Ambiguity

This kind of ambiguity occurs when a sentence is parsed in different ways. For example, the sentence “The man saw the girl with the telescope”. It is ambiguous whether the man saw the girl carrying a telescope or he saw her through his telescope.

Semantic Ambiguity

This kind of ambiguity occurs when the meaning of the words themselves can be misinterpreted. In other words, semantic ambiguity happens when a sentence contains an ambiguous word or phrase. For example, the sentence “The car hit the pole while it was

moving” is having semantic ambiguity because the interpretations can be “The car, while moving, hit the pole” and “The car hit the pole while the pole was moving”.

Anaphoric Ambiguity

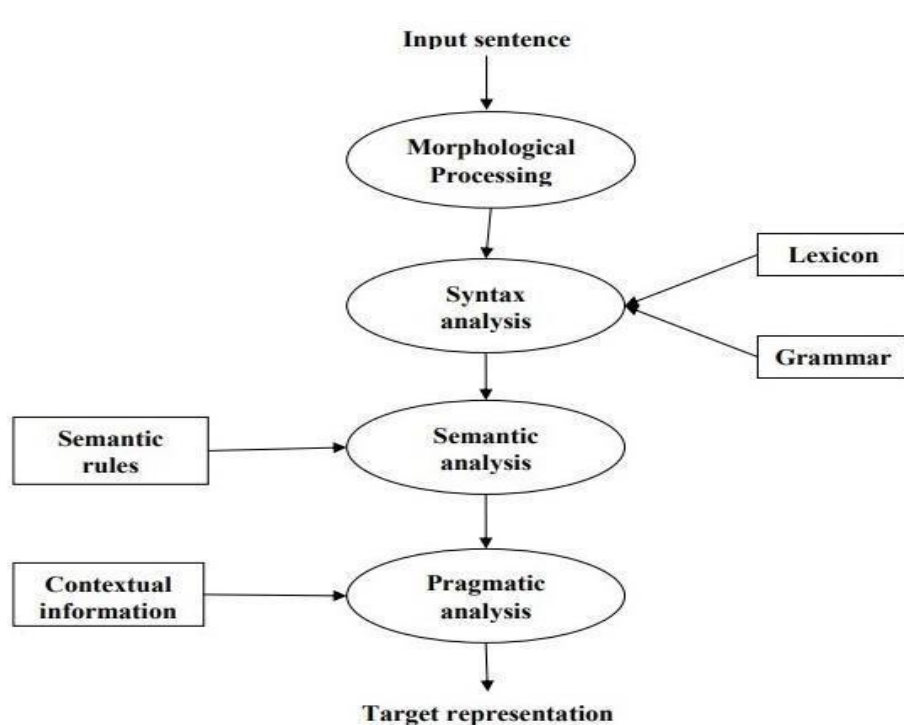
This kind of ambiguity arises due to the use of anaphora entities in discourse. For example, the horse ran up the hill. It was very steep. It soon got tired. Here, the anaphoric reference of “it” in two situations cause ambiguity.

Pragmatic ambiguity

Such kind of ambiguity refers to the situation where the context of a phrase gives it multiple interpretations. In simple words, we can say that pragmatic ambiguity arises when the statement is not specific. For example, the sentence “I like you too” can have multiple interpretations like I like you (just like you like me), I like you (just like someone else dose).

NLP Phases

Following diagram shows the phases or logical steps in natural language processing –



Morphological Processing or Lexical analysis

It is the first phase of NLP. The purpose of this phase is to break chunks of language input into sets of tokens corresponding to paragraphs, sentences and words.

In this Phase ,the source code is scanned as a stream of characters and converted into intelligible lexemes. The entire book is divided into paragraphs, phrases, and words. It searches for morphemes, which are the smallest units of a word.

For example, a word like **“uneasy”** can be broken into two sub-word tokens as **“un-easy”**.

Syntax Analysis

It is the second phase of NLP. It is a technique for checking grammar, arranging words, and displaying relationships between them.

The purpose of this phase is two folds:

- to check that a sentence is well formed or not and
- to break it up into a structure that shows the syntactic relationships between the different words.

Syntax analysis guarantees that the structure of a particular piece of text is proper.

For example, the sentence like **“The school goes to the boy”** would be rejected by syntax analyzer or parser as it makes no sense.

Semantic Analysis

It is the third phase of NLP. The purpose of this phase is to draw exact meaning, or you can say dictionary meaning from the text. The text is checked for meaningfulness. It also deals with putting words together to form sentences. It extracts the text's exact meaning or dictionary definition. The meaning of the text is examined. It is accomplished by mapping the task domain's syntactic structures and objects.

For example, semantic analyzer would reject a sentence like **“Hot ice-cream”** yet it is illogical because ice-cream cannot be hot.

Pragmatic Analysis

It is the fourth phase of NLP. Pragmatic Analysis uses a set of rules that describe cooperative dialogues to help you find the intended result.

It covers things like word repetition, who said what to whom, and so on. It comprehends how people communicate with one another, the context in which they converse, and a variety of other factors. It refers to the process of abstracting or extracting the meaning of a situation's use of language. It translates the given text using the knowledge gathered in the preceding stages.

For example, the sentence “Put the banana in the basket on the shelf” can have two semantic interpretations and pragmatic analyzer will choose between these two possibilities.

Regular Expressions

A regular expression (RE) is a language for specifying text search strings. RE helps us to match or find other strings or sets of strings.

Regular expressions are used to search texts in UNIX as well as in MS WORD in identical way. We have various search engines using a number of RE features.

Properties of Regular Expressions

Mathematically, A Regular Expression can be defined as follows –

ϵ is a Regular Expression, which indicates that the language is having an empty string.

ϕ is a Regular Expression which denotes that it is an empty language.

If X and Y are Regular Expressions, then

- X, Y
- $X.Y$ (Concatenation of XY)
- $X+Y$ (Union of X and Y)
- X^*, Y^* (Closure of X and Y)

are also regular expressions.

If a string is derived from above rules then that would also be a regular expression.

Examples of Regular Expressions

The following table shows a few examples of Regular Expressions – Regular Expressions

Regular Set

$(a + b)$	$\{a, b\}$
$(a.b)$	$\{ab\}$
(a^*)	$\{\epsilon, a, aa, aaa, aaaa, \dots\}$

Regular Sets & Their Properties

It may be defined as the set that represents the value of the regular expression and consists specific properties.

Properties of regular sets

If we do the union of two regular sets then the resulting set would also be regular. If we do the intersection of two regular sets then the resulting set would also be regular.

If we do the complement of regular sets, then the resulting set would also be regular. If we do the difference of two regular sets, then the resulting set would also be regular.

If we do the reversal of regular sets, then the resulting set would also be regular.

If we take the closure of regular sets, then the resulting set would also be regular. If we do the concatenation of two regular sets, then the resulting set would also be regular.

Finite State Automata

The term automata, derived from the Greek word "αὐτόματα" meaning "self-acting", is the plural of automaton which may be defined as an abstract self-propelled computing device that follows a predetermined sequence of operations automatically.

An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).

Mathematically, an automaton can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states.

Σ is a finite set of symbols, called the alphabet of the automaton.

δ is the transition function

q_0 is the initial state from where any input is processed ($q_0 \in Q$). F is

a set of final state/states of Q ($F \subseteq Q$).

Relation between Finite Automata, Regular Grammars and Regular Expressions

Following points will give us a clear view about the relationship between finite automata, regular grammars and regular expressions –

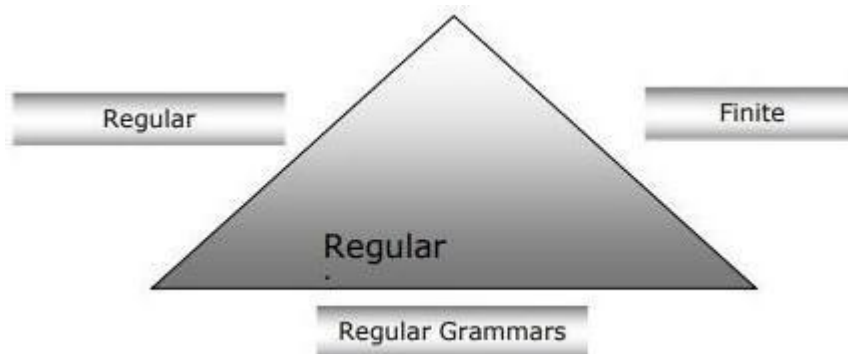
As we know that finite state automata are the theoretical foundation of computational work and regular expressions is one way of describing them.

We can say that any regular expression can be implemented as FSA and any FSA can be described with a regular expression.

On the other hand, regular expression is a way to characterize a kind of language called regular language. Hence, we can say that regular language can be described with the help of both FSA and regular expression.

Regular grammar, a formal grammar that can be right-regular or left-regular, is another way to characterize regular language.

Following diagram shows that finite automata, regular expressions and regular grammars are the equivalent ways of describing regular languages.



Types of Finite State Automation (FSA)

Finite state automation is of two types. Let us see what the types are.

Deterministic Finite automation (DFA)

It may be defined as the type of finite automation wherein, for every input symbol we can determine the state to which the machine will move. It has a finite number of states that is why the machine is called Deterministic Finite Automaton (DFA).

Mathematically, a DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states.

Σ is a finite set of symbols, called the alphabet of the automaton. δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$.

q_0 is the initial state from where any input is processed ($q_0 \in Q$). F is a set of final state/states of Q ($F \subseteq Q$).

Whereas graphically, a DFA can be represented by diagrams called state diagrams where –

The states are represented by **vertices**. The transitions are shown by labeled **arcs**.

The initial state is represented by an **empty incoming arc**. The final state is represented by **double circle**.

Example of DFA

Suppose a DFA be

$$Q = \{a, b, c\},$$

$$\Sigma = \{0, 1\},$$

$$q_0 = \{a\},$$

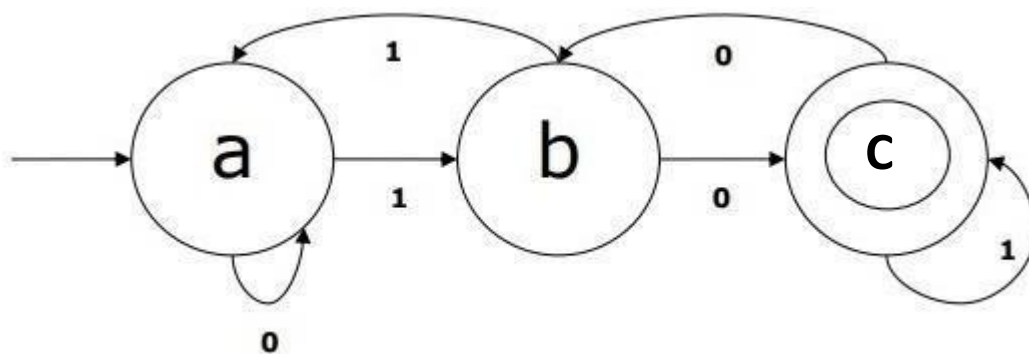
$$F = \{c\},$$

Transition function δ is shown in the table as follows –

Current State	Next State for Input 0	Next State for Input 1
---------------	------------------------	------------------------

A	a	B
B	c	A
C	b	C

The graphical representation of this DFA would be as follows –



Non-deterministic Finite Automaton (NFA)

It may be defined as the type of finite automation where for every input symbol we cannot determine the state to which the machine will move i.e. the machine can move to any combination of the states. It has a finite number of states that is why the machine is called Non-deterministic Finite Automaton (NFA).

Mathematically, NFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where – Q is a finite set of states.

Σ is a finite set of symbols, called the alphabet of the automaton. δ :-

is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$.

q_0 :- is the initial state from where any input is processed ($q_0 \in Q$). F :-

is a set of final state/states of Q ($F \subseteq Q$).

Whereas graphically (same as DFA), a NFA can be represented by diagrams called state diagrams where –

The states are represented by **vertices**. The transitions are shown by labeled **arcs**.

The initial state is represented by an **empty incoming arc**. The final state is represented by double **circle**.

Example of NDFA

Suppose a NDFA be

$Q = \{a, b, c\}$,

$\Sigma = \{0, 1\}$,

$q_0 = \{a\}$,

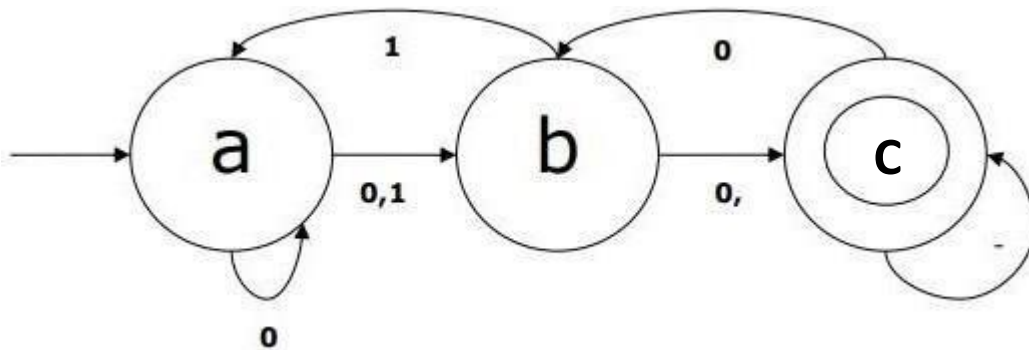
$F = \{c\}$,

Transition function δ is shown in the table as follows –

Current State Next State for Input 0 Next State for Input 1

A	a, b	B
B	C	a, c
C	b, c	C

The graphical representation of this NDFA would be as follows –



Morphological Parsing

The term morphological parsing is related to the parsing of morphemes. We can define morphological parsing as the problem of recognizing that a word breaks down into smaller meaningful units called morphemes producing some sort of linguistic structure for it. For example, we can break the word *foxes* into two, *fox* and *-es*. We can see that the word *foxes*, is made up of two morphemes, one is *fox* and other is *-es*.

In other sense, we can say that morphology is the study of –

The formation of words.

The origin of the words.

Grammatical forms of the words.

Use of prefixes and suffixes in the formation of words. How parts-of-speech (PoS) of a language are formed.

Types of Morphemes

Morphemes, the smallest meaning-bearing units, can be divided into two types – Stems
Word Order

Stems

It is the core meaningful unit of a word. We can also say that it is the root of the word. For example, in the word foxes, the stem is fox.

Affixes – As the name suggests, they add some additional meaning and grammatical functions to the words. For example, in the word foxes, the affix is – es.

Further, affixes can also be divided into following four types –

- **Prefixes** – As the name suggests, prefixes precede the stem. For example, in the word unbuckle, un is the prefix.
- **Suffixes** – As the name suggests, suffixes follow the stem. For example, in the word cats, -s is the suffix.
- **Infixes** – As the name suggests, infixes are inserted inside the stem. For example, the word cupful, can be pluralized as cupsful by using -s as the infix.
- **Circumfixes** – They precede and follow the stem. There are very less examples of circumfixes in English language. A very common example is ‘A- ing’ where we can use -A precede and -ing follows the stem.

Word Order

The order of the words would be decided by morphological parsing. Let us now see the requirements for building a morphological parser –

Lexicon

The very first requirement for building a morphological parser is lexicon, which includes the list of stems and affixes along with the basic information about them. For example, the information like whether the stem is Noun stem or Verb stem, etc.

Morphotactics

It is basically the model of morpheme ordering. In other sense, the model explaining which classes of morphemes can follow other classes of morphemes inside a word. For example, the morphotactic fact is that the English plural morpheme always follows the noun rather than preceding it.

Orthographic rules

These spelling rules are used to model the changes occurring in a word. For example, the rule of converting y to ie in word like city+s = cities not citys.

Concept of Grammar

Grammar is very essential and important to describe the syntactic structure of well-formed programs. In the literary sense, they denote syntactical rules for

conversation in natural languages. Linguistics have attempted to define grammars since the inception of natural languages like English, Hindi, etc.

The theory of formal languages is also applicable in the fields of Computer Science mainly in programming languages and data structure. For example, in 'C' language, the precise grammar rules state how functions are made from lists and statements.

A mathematical model of grammar was given by **Noam Chomsky** in 1956, which is effective for writing computer languages.

Mathematically, a grammar G can be formally written as a 4-tuple (N, T, S, P) where

N or **V_N** = set of non-terminal symbols, i.e., variables.

T or Σ = set of terminal symbols.

S = Start symbol where $S \in N$

P denotes the Production rules for Terminals as well as Non-terminals. It has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \Sigma$ and least one symbol of α belongs to V_N

Phrase Structure or Constituency Grammar

Phrase structure grammar, introduced by Noam Chomsky, is based on the constituency relation. That is why it is also called constituency grammar. It is opposite to dependency grammar.

Example

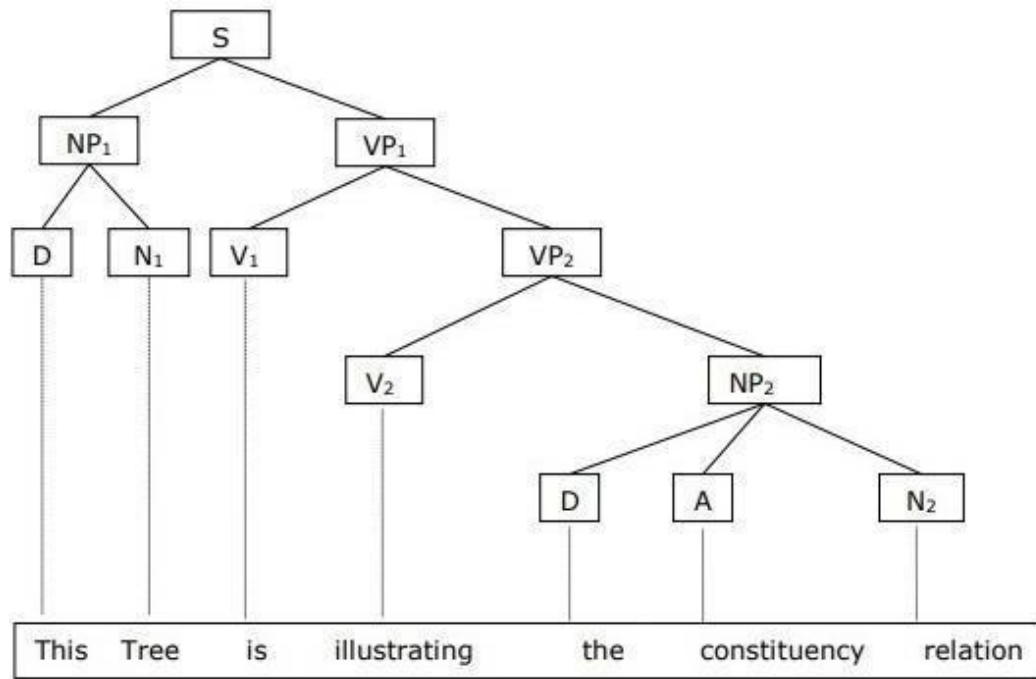
Before giving an example of constituency grammar, we need to know the fundamental points about constituency grammar and constituency relation.

All the related frameworks view the sentence structure in terms of constituency relation.

The constituency relation is derived from the subject-predicate division of Latin as well as Greek grammar.

The basic clause structure is understood in terms of **noun phrase NP** and **verb phrase VP**.

We can write the sentence “**This tree is illustrating the constituency relation**” as follows –



Dependency Grammar

It is opposite to the constituency grammar and based on dependency relation. It was introduced by Lucien Tesnière. Dependency grammar (DG) is opposite to the constituency grammar because it lacks phrasal nodes.

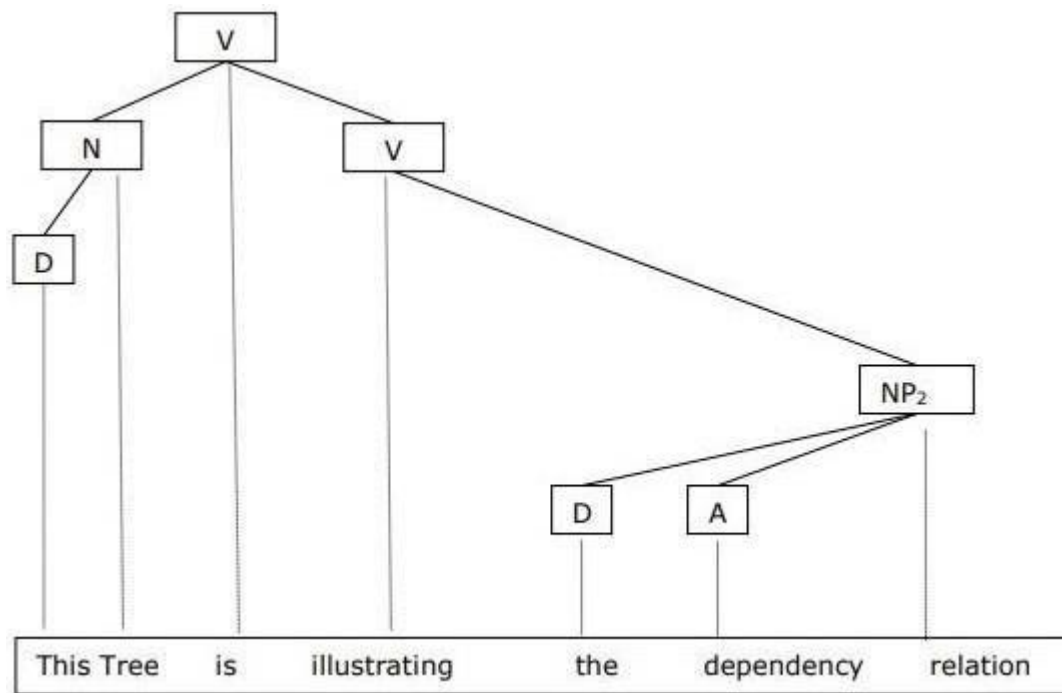
Example

Before giving an example of Dependency grammar, we need to know the fundamental points about Dependency grammar and Dependency relation.

In DG, the linguistic units, i.e., words are connected to each other by directed links. The verb becomes the center of the clause structure.

Every other syntactic units are connected to the verb in terms of directed link. These syntactic units are called *dependencies*.

We can write the sentence “**This tree is illustrating the dependency relation**” as follows;



Parse tree that uses Constituency grammar is called constituency-based parse tree; and the parse trees that uses dependency grammar is called dependency-based parse tree.

Context Free Grammar

Context free grammar, also called CFG, is a notation for describing languages and a superset of Regular grammar. It can be seen in the following diagram –



Definition of CFG

CFG consists of finite set of grammar rules with the following four components –

Set of Non-terminals

It is denoted by V . The non-terminals are syntactic variables that denote the sets of strings, which further help defining the language, generated by the grammar.

Set of Terminals

It is also called tokens and defined by Σ . Strings are formed with the basic symbols of terminals.

Set of Productions

It is denoted by P. The set defines how the terminals and non-terminals can be combined. Every production(P) consists of non-terminals, an arrow, and terminals (the sequence of terminals). Non-terminals are called the left side of the production and terminals are called the right side of the production.

Start Symbol

The production begins from the start symbol. It is denoted by symbol S. Non-terminal symbol is always designated as start symbol.

Studying meaning of individual word

It is the first part of the semantic analysis in which the study of the meaning of individual words is performed. This part is called lexical semantics.

Studying the combination of individual words

In the second part, the individual words will be combined to provide meaning in sentences.

The most important task of semantic analysis is to get the proper meaning of the sentence. For example, analyze the sentence “**Ram is great.**” In this sentence, the speaker is talking either about Lord Ram or about a person whose name is Ram. That is why the job, to get the proper meaning of the sentence, of semantic analyzer is important.

Types of Language Models

- **Statistical Models:**

Statistical models develop probabilistic models that help with predictions for the next word in thesequence. It also uses data to make predictions depending on the words that preceded. Moreover, there are multiple statistical language models that help businesses. For instance, N-Gram, Unigram, Bidirectional, exponential, etc are all examples of statistical models.

- **Neural Language Models:**

Neural Language Models refer to language models that are developed using neural networks. Moreover, the models help mitigate the challenges that occur in classical language models. Further, it helps execute complex tasks like speech recognition or machine transition.

Examples of Language Models

- **Speech Recognition:**

Firstly, voice assistants like Siri, Alexa, Google Homes, etc. are the biggest examples of the way language models support machines in processing speech and audio commands.

- **Machine Translation:**

Further, Google Translator and Microsoft Translate are examples of language models helping machines to translate words and text to various languages.

- **Sentiment analysis:**

Sentiment analysis is the process of identifying sentiments and behaviors on the basis of the text. Further, NLP Models helps businesses to recognize their customer's intentions and attitude using text. For example, Hubspot's Service Hub analyzes sentiments and emotions using NLP language models.

- **Parsing Tools:**

Parsing refers to analyzing sentences and words that are complementary according to syntax and grammar rules. Further, language models enable features like spell-checking.

- **Optical Character Recognition (OCR):**

OCR is the use of machines to transform images of text into machine-encoded text. Moreover, the image may be converted from a scanned document or picture. It is also an important function that helps digitize old paper trails. Hence, it helps analyze and identify handwriting samples.

- **Information Retrieval:**

It refers to searching documents and files for information. It also includes regular searches for documents and files and probing for metadata that leads to a document.

Statistical Language Modeling

What is statistical language modeling in NLP?

Statistical Language Modeling, or Language Modeling and LM for short, is the development of probabilistic models that can predict the next word in the sequence given the words that precede it.

A statistical language model learns the probability of word occurrence based on examples of text. Simpler models may look at a context of a short sequence of words, whereas larger models may work at the level of sentences or paragraphs. Most commonly, language models operate at the level of words.

You could develop a language model and use it standalone for purposes like generating new sequences of text that appear to have come from the body.

Language modeling is a core problem for a rather wide range of natural language processing tasks. Language models are generally used on the front-end or back-end of a more sophisticated model for a task that needs language understanding.

What are the types of statistical language models?

Statistical models include the development of probabilistic models that are able to predict the next word in the sequence, given the words that precede it. A number of statistical language models are in use already. Let's take a look at some of those popular models:

1. N-Gram

This is one of the simplest approaches to language modelling. Here, a probability distribution for a sequence of 'n' is created, where 'n' can be any number and defines the size of the gram (or sequence of words being assigned a probability). If $n=4$, a gram may look like: "can you help me". Basically, 'n' is the amount of context that the model is trained to consider. There are different types of N-Gram models such as unigrams, bigrams, trigrams, etc.

2. Exponential

This type of statistical model evaluates text by using an equation which is a combination of n-grams and feature functions. Here the features and parameters of the desired results are already specified. The model is based on the principle of entropy, which states that probability distribution with the most entropy is the best choice. Exponential models have fewer statistical assumptions which mean the chances of having accurate results are more.

3. Continuous Space

In this type of statistical model, words are arranged as a non-linear combination of weights in a neural network. The process of assigning weight to a word is known as wordembedding. This type of model proves helpful in scenarios where the data set of words continues to become large and include unique words.

In cases where the data set is large and consists of rarely used or unique words, linear models such as n-gram do not work. This is because, with increasing words, the possible word sequences increase, and thus the patterns predicting the next word become weaker.

How do you build a simple Statistical Language Model?

Language models start with a Markov Assumption. This is a simplifying assumption that the $k+1$ st word is dependent on the previous k words. A 2nd order assumption results in a Bigram model. The models are training using Maximum Likelihood Estimations (MLE) of an existing corpus. The MLE approach then is simply a fraction of word counts.

There are some advantages of using tradition n-gram language models.

- They are easy to train on a large corpus
- They work surprisingly well in most tasks!!
- However, they have some disadvantages

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}})$$

prob of a n-gram $\rightarrow P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})$

prob of a (n-1)-gram $\rightarrow P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})$

Source: Feedly Blog

What are the applications of statistical language modeling?

Statistical language models are used to generate text in many similar natural language processing tasks, such as:

Speech Recognition

Voice assistants such as Siri and Alexa are examples of how language models help machines in processing speech audio.

Machine Translation

Google Translator and Microsoft Translate are examples of how NLP models can help in translating one language to another.

Sentiment Analysis

This helps in analyzing sentiments behind a phrase. This use case of NLP models is used in products that allow businesses to understand a customer's intent behind opinions or attitudes expressed in the text. Hubspot's Service Hub is an example of how language models can help in sentiment analysis.

Text Suggestions

Google services such as Gmail or Google Docs use language models to help users get text suggestions while they compose an email or create long text documents, respectively.

Parsing Tools

Parsing involves analyzing sentences or words that comply with syntax or grammar rules. Spell checking tools are perfect examples of language modelling and parsing.

Language models are also used to generate text in other similar language processing tasks like optical character recognition, handwriting recognition, image captioning, etc.

What are the drawbacks of statistical language modeling?

1. Zero probabilities

If we have a tri-gram language model that conditions on two words and has a vocabulary of 10,000 words. Then we have 10^{12} triplets. If our training data has 10^{10} words, there are many triples that will never be observed in the training data and thus the basic MLE will assign zero probabilities to those events. And a zero-probability translates to infinite perplexity. To overcome this issue many techniques have been developed under the family of Smoothing Techniques. A good overview of these techniques is presented in this paper.

2. Exponential Growth

The second challenge is that the number of n-grams grows as an nth exponent of the vocabulary size. A 10,000-word vocabulary would have 10^{12} tri-grams and a 100,000 word vocabulary will have 10^{15} trigrams.

3. Generalization

The last issue with MLE techniques is the lack of generalization. If the model sees the term 'white horse' in the training data but does not see 'black horse', the MLE will assign zero probability to 'black horse'. (Thankfully, it will assign zero probability to Purple horse as well)

CLASSIFICATION OF LANGUAGE MODELS

Language models (LM) can be classified into two categories:

1. **Count-based LMs / Conventional Statistical Language Models:** The count-based methods, such as traditional statistical models, usually involve making an n-th order Markov assumption and estimating n-gram probabilities via counting and subsequent smoothing. These models use traditional statistical techniques like N-grams, Hidden Markov Models (HMM) and certain linguistic rules to learn the probability distribution of words
2. **Continuous-space LMs / Neural Language Models:** Continuous-space LM is also known as *Neural language model* (NLM). They use different kinds of Neural Networks to model language. In recent years, LMs developed in this category are Feed-forward neural probabilistic language models (NPLMs) and Recurrent Neural Network language models (RNNs).

Count-based / Conventional Statistical Language models

Statistical models include the development of probabilistic models that are able to predict the next word in the sequence, given the words that precede it. A few popular models under this category are listed below:

- **N-Gram:** This is one of the simplest approaches to language modelling. Here, a probability distribution for a sequence of 'n' is created, where 'n' can be any number and defines the size of the gram (or sequence of words being assigned a probability). If n=4, a gram may look like: "can you help me". Basically, 'n' is the amount of context that the model is trained to consider. There are different types of N-Gram models such as unigrams, bigrams, trigrams, etc.
- **Exponential:** This type of statistical model evaluates text by using an equation which is a *combination of n-grams and feature functions*. Here the features and parameters of the desired results are already specified. The model is based on the *principle of entropy*, which states that probability distribution with the most entropy is the best choice. Exponential models have fewer statistical assumptions which mean the chances of having accurate results are more.

UNIT II

WORD LEVEL ANALYSIS

Unsmoothed N-grams, Evaluating N-grams, Smoothing, Interpolation and Backoff – Word Classes, Part-of-Speech Tagging, Rule-based, Stochastic and Transformation-based tagging, Issues in PoS tagging – Hidden Markov and Maximum Entropy models.

Part of Speech (PoS) Tagging

Tagging is a kind of classification that may be defined as the automatic assignment of a description to the tokens. Here the descriptor is called tag, which may represent one of the part-of-speech, semantic information and so on.

Now, if we talk about Part-of-Speech (PoS) tagging, then it may be defined as the process of assigning one of the parts of speech to the given word. It is generally called POS tagging. In simple words, we can say that POS tagging is a task of labelling each word in a sentence with its appropriate part of speech. We already know that parts of speech include nouns, verb, adverbs, adjectives, pronouns, conjunction and their sub-categories.

Most of the POS tagging falls under Rule Base POS tagging, Stochastic POS tagging and Transformation based tagging.

Rule-based POS Tagging

One of the oldest techniques of tagging is rule-based POS tagging. Rule-based taggers use dictionary or lexicon for getting possible tags for tagging each word. If the word has more than one possible tag, then rule-based taggers use hand-written rules to identify the correct tag. Disambiguation can also be performed in rule-based tagging by analyzing the linguistic features of a word along with its preceding as well as following words. For example, suppose if the preceding word of a word is article then word must be a noun.

As the name suggests, all such kind of information in rule-based POS tagging is coded in the form of rules. These rules may be either –

1. Context-pattern rules
2. Or, as Regular expression compiled into finite-state automata, intersected with lexically ambiguous sentence representation.

We can also understand Rule-based POS tagging by its two-stage architecture –

First stage – In the first stage, it uses a dictionary to assign each word a list of potential parts-of-speech.

Second stage – In the second stage, it uses large lists of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word.

Properties of Rule-Based POS Tagging

Rule-based POS taggers possess the following properties – These taggers are knowledge-driven taggers.

The rules in Rule-based POS tagging are built manually.

The information is coded in the form of rules.

We have some limited number of rules approximately around 1000.

Smoothing and language modeling is defined explicitly in rule-based taggers.

Stochastic POS Tagging

Another technique of tagging is Stochastic POS Tagging. Now, the question that arises here is which model can be stochastic. The model that includes frequency or probability (statistics) can be called stochastic. Any number of different approaches to the problem of part-of-speech tagging can be referred to as stochastic tagger.

The simplest stochastic tagger applies the following approaches for POS tagging –

Word Frequency Approach

In this approach, the stochastic taggers disambiguate the words based on the probability that a word occurs with a particular tag. We can also say that the tag encountered most frequently with the word in the training set is the one assigned to an ambiguous instance of that word. The main issue with this approach is that it may yield inadmissible sequence of tags.

Tag Sequence Probabilities

It is another approach of stochastic tagging, where the tagger calculates the probability of a given sequence of tags occurring. It is also called n-gram approach. It is called so because the best tag for a given word is determined by the probability at which it occurs with the n previous tags.

Properties of Stochastic POST Tagging

Stochastic POS taggers possess the following properties –

- This POS tagging is based on the probability of tag occurring. It requires training corpus

- There would be no probability for the words that do not exist in the corpus. It uses different testing corpus (other than training corpus).

- It is the simplest POS tagging because it chooses most frequent tags associated with a word in training corpus.

Transformation-based Tagging

Transformation based tagging is also called Brill tagging. It is an instance of the transformation-based learning (TBL), which is a rule-based algorithm for automatic tagging of POS to the given text. TBL, allows us to have linguistic knowledge in a readable form, transforms one state to another state by using transformation rules.

It draws the inspiration from both the previous explained taggers – rule-based and stochastic. If we see similarity between rule-based and transformation tagger, then like rule-based, it is also based on the rules that specify what tags need to be assigned to what words. On the other hand, if we see similarity between stochastic and transformation tagger then like stochastic, it is machine learning technique in which rules are automatically induced from data.

Methods of Language Modelings:

Two types of Language Modelings:

- **Statistical Language Modelings:** Statistical Language Modeling, or Language Modeling, is the development of probabilistic models that are **able to predict the next word in the sequence given the words that precede. Examples such as N-gram language modeling.**
- **Neural Language Modelings:** Neural network methods are achieving better results than classical methods both on standalone language models and when models are incorporated into larger models on challenging tasks like speech recognition and machine translation. A way of performing a neural language model is through word embeddings.

N-gram

N-gram can be defined as the contiguous sequence of n items from a given sample of text or speech. The items can be letters, words, or base pairs according to the application. The N-grams typically are collected from a text or speech corpus (A long text dataset).

N-gram Language Model:

An N-gram language model predicts the probability of a given N-gram within any sequence of words in the language. A good N-gram model can predict the next word in the sentence i.e the value of $p(w|h)$

Example of N-gram such as unigram (“This”, “article”, “is”, “on”, “NLP”) or bi-gram (“This article”, “article is”, “is on”, “on NLP”).

Now, we will establish a relation on how to find the next word in the sentence using

. We need to calculate $p(w|h)$, where w is the candidate for the next word. For example in the above example, let's consider, we want to calculate what is the probability of the last word being “NLP” given the previous words:

$$p(NLP|this\ article\ is\ on)$$

After generalizing the above equation can be calculated as:

$$p(w_5|w_1, w_2, w_3, w_4) \text{ or } P(W)$$

$$= p(w_n|w_1, w_2, \dots, w_n)$$

But how do we calculate it? The answer lies in the chain rule of probability:

$$P(A|B) = \frac{P(A,B)}{P(B)}$$
$$P(A, B) = P(A|B)P(B)$$

Now generalize the above equation:

$$P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2) \dots P(X_n|X_1, X_2, \dots, X_{n-1})$$
$$P(w_1 w_2 w_3 \dots w_n) = \prod_i P(w_i | w_1 w_2 \dots w_{i-1})$$

Simplifying the above formula using Markov assumptions:

For unigram:

$$\frac{P(w_1, w_2, \dots, w_n)}{P(w_1)P(w_2)\dots P(w_n)} \approx \frac{P(w_i | w_{i-k}, \dots, w_{i-1})}{P(w_i)}$$

- **For Bigram:**

$$P(w_i | w_1 w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-1})$$

Evaluating N-grams

Metrics for Language Modelings

- **Entropy:** Entropy, as a measure of the amount of information conveyed by Claude Shannon. Below is the formula for representing entropy

$$H(p) = \sum_x p(x) \cdot (-\log(p(x)))$$

- **Cross-Entropy:** It measures the ability of the trained model to represent test data W_1^{i-1} .

$$H(p) = \sum_{i=1}^x \frac{1}{n} (-\log_2(p(w_i | w_1^{i-1})))$$

The cross-entropy is always greater than or equal to Entropy i.e the model uncertainty can be no less than the true uncertainty.

- **Perplexity:** Perplexity is a measure of how good a probability distribution predicts a sample. It can be understood as a measure of uncertainty. The perplexity can be calculated by cross-entropy to the exponent of 2.

$$2^{\text{Cross-Entropy}}$$

Following is the formula for the calculation of Probabi
the language model, normalized by the number of wor

$$PP(W) = \sqrt[n]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

For Example:

- Let's take an example of the sentence: **'Natural Language Processing'** predicting the first word, let's say the word has the

word	P(word <start>)
The	0.4
Processing	0.3

Natural	0.12
Language	0.18

Now, we know the probability of getting the first word as natural. But, what's the probability of getting the next word after getting the word 'Language' after the word 'Natural'.

word	P(word 'Natural')
The	0.05
Processing	0.3
Natural	0.15
Language	0.5

Interpolation and Backoff

In unigram prior smoothing, we replace $(1/V)$ with $P(W_i)$. This includes Interpolation as we include Unigram result in Bigram calculation. (Read Interpolation below). This works well but still cannot be used for Language Modelling.

Add one smoothing makes massive changes in actual data. It's used in text-classification where the number of zeros are not large.

Backoff : Sometimes we don't have enough data to trust our k-model, so we move to lower model Language model.

Interpolation: Mix Unigram, Bigram and Trigram. Linear Interpolation: It is of 2 types.

1. Simple Interpolation: $L_1 P(W_i) + L_2 P(W_i|W_{i-1}) + L_3 P(W_i|W_{i-2}W_{i-1})$; $L_1+L_2+L_3 = 1$;
2. Lambda conditional Interpolation: Lambdas depend on context.

The value of these lambdas can be calculated if we have a dev set/training data to set these lambdas to maximize the likelihood of these models.

Open vs Closed Vocabulary tasks: If we know all the words in advance then it's a closed Vocabulary. But many times we don't know all words. In our test set there might occur OOV (Out of Vocabulary) words. One way to deal with them is to create a special token called <UNK>. Any word which has low probability while training is changed to <UNK> and then the model is treated as if the word <UNK> is there. At decoding time, we change the unseen word to <UNK> and then compute its probability w.r.t the Language Model.

Smoothing for Large N-Grams (Web-scale-N-Grams): We use **Stupid Backoff** technique. $P(W_i|W_{i-k} \dots W_{i-1}) = \{ \text{simple probability formula} \}$ but if $\text{count}(W_{i-k} \dots W_{i-1})$ is less than some threshold then we use $k * P(W_i|W_{i-k+1} \dots W_{i-1})$. Where k is some fraction and this

can be best calculated by our dev set data. **Stupid backoff** produces scores rather than probabilities. And this works quite well for large scale N-Grams.

Add-K does not work good for Language modelling. Stupid backoff works well for Large N-Grams. Let's start with Good Turing smoothing.

Good Turing Smoothing

N_c = Frequency of frequency c . Example : Abhinav I am Abhinav am I this this = 1 ; I = 2 ; am = 2 ;
Abhinav 2
 $N_1 = 1$; $N_2 = 3$

Read the intuition of Good Turing smoothing [here](#). Done by leave one out validation and by taking a held set. Very well explained [here](#) (from 11th Minute).

$P(\text{new things occurring}) =$

N_1 / N ; Generalizing: $c^* = (c+1) * N_{c+1} / N_c$ And $P = c^* / N$.

Therefore in case of new things, $c^* = 0$ as we have not observed it earlier. After first few N 's namely $N_0, N_1 \dots N_{10}$, we replace the N 's by a smooth function as there will be gaps. Say N_{127} can be zero. Therefore we replace with the best fit paralog function.

Kneser Ney Smoothing

First do absolute discounting (as a result of good turing smoothing). But we see the discounting factor is nearly equal to 0.75. So we do absolute discounting and then do interpolation — Discounting + $(L(W_{i-1}) * P(w))$.

Instead of $P(w)$ (Unigram probability) {How likely is w }, $P_{\text{continuation}}(w)$ {How likely is w to appear as a novel continuation} is a better estimate.

$P_{\text{continuation}}(w) = \text{Words which precede } w / \text{Total word bigrams}$.

A frequent word like Francisco will have low continuation probability as it only appears after San. $P_{\text{kn}}(W_i | W_{i-1}) = (\text{count}(W_{i-1}W_i) - d) / (\text{count}(W_{i-1}) + L(W_{i-1})P_{\text{cont}}(W_i))$

Kneser Ney Smoothing for N-Grams is as below:

$P_{\text{kn}}(w_i | w_{i-n+1} \dots w_{i-1}) = [\max(\text{count}_{\text{kn}}(w_{i-n+1} \dots w_i) - d, 0)] / [\text{count}_{\text{kn}}(w_{i-n+1} \dots w_{i-1}) + \Theta(w_{i-n+1} \dots w_{i-1}) \times P_{\text{kn}}(w_i | w_{i-n+2} \dots w_{i-1})]$

Where:

$\text{ckn}(\bullet) = \text{the actual count}(\bullet) \text{ for the highest order } n\text{-gram}$

· the actual count($\bullet$) for the highest order n -gram

or

· $\text{continuation_count}(\bullet)$ for lower order n -gram

$\Rightarrow \text{continuation_count} = \text{Number of unique single word contexts for}$

Word Classes

Words can be grouped into classes referred to as Part of Speech (PoS) or morphological classes

Traditional grammar is based on few types of PoS (noun, verb, adjective, preposition, adverb, conjunction, etc..) More recent models are based on a larger number of classes

45 Penn Treebank

87 Brown corpus

146 C7 tagset

The word PoS provides crucial information to determine the roles of the word itself and of the words close to it in the sentence

knowing if a word is a personal pronoun (I, you, he/she,...) or a possessive pronoun (my, your, his/her,...) allows a more accurate selection of the most probable words that appear in its neighborhood (the syntactic rules are often based on the PoS of words)

e.g. possessive pronoun - noun vs. personal pronoun – verb

The 4 largest open classes of words, present in most of the languages, are

- nouns
- verbs
- adverbs
- adjectives

Nouns are concrete terms (e.g. ship, table), abstractions (e.g. relationship, function), verb-like terms (e.g. pacing, pricing)

- They can be functionally tied to determiners (the ship, a ship, ..) and they can assume the plural form (the ships), etc..
- They are traditionally divided into proper nouns (e.g. Marco, Italy) and common nouns (e.g. book, lecture)
- In many languages common nouns are also divided count nouns (they have the plural form) and mass nouns (they are used only in their singular form, e.g. snow, communism)

Word classes- verbs & adjectives

- The class of verbs includes most of the words that refer to actions and processes
 - to write, to go, to eat
 - they have “some” morphological inflections
 - In English non-3rd-person-sg (eat), 3rd-person-sg (eats), progressive (eating), past-participle (eaten), past perfect (ate)
 - A special class of verbs is that of auxiliary verbs (to be, to have)
- The class of adjectives contains terms describing properties or qualities
 - Most of the languages have adjectives for concepts like color (white, red,...), age (young, old,...) quality (good, bad,...), etc..

Word classes- adverbs

- Usually adverbs are used to modify other terms (not only verbs)
 - Directional or locative adverbs specify the direction or location of a given action (here, there, up, ..)
 - Degree adverbs specify the extent of an action, process or property (extremely, very,...)
 - Manner adverbs describe the modality of some action or process (slowly, delicately, smartly,...)
 - Temporal adverbs describe the time for an action or event (yesterday, today, before, after, later, Monday,...)
- The class of adverbs is somehow heterogeneous
 - Some adverbs are similar to nouns (e.g. Monday – we will meet Monday, we meet on Mondays)

PoS tagging & tags

- PoS tagging consists in assigning a tag to each word in a document
 - The selection of the employed tagset depends on the language and specific application
 - The input is a word sequence and the employed tagset while the output is the association of each word to its “best” tag
 - There may exist more tags for a given word (ambiguity)

Did/VBD you/PP box/??? your/PP\$ belongings/NNS ?/?
/NN /VB /VBP

- The PoS tagger task is to solve these ambiguities by selecting the most appropriate tag given the word context
 - The percentage of ambiguous words is not too high, but among them there are very frequent words (e.g. *can* – Auxiliary verb, Noun, Verb, *still* has 7 compatible tags – adj, adv, verb, noun)

PoS tagging algorithms

- Rule-based taggers
 - They are based on the “handcrafting” of a large rule-base that specifies the conditions to be verified to assign a specific tag in the ambiguous cases
 - e.g. a word is a noun if it is preceded by a determiner
- Probabilistic taggers
 - They resolve ambiguities by estimating the probability that a given word is a specific tag in the observed context. The parameters for the probability model are estimated on a reference corpus.
- Other approaches
 - tagging can be cast as a classification task (each tag corresponds to a class and the classifiers exploits features that describe the context – e.g. features of the words on the left/right of the considered word)
 - Taggers can exploits rules learnt from examples

Rule-based PoS tagging

- Two step process (e.g. ENGTWOL Voutilainen, 1995)
 - Word tagging using a lexicon (more than one tag can be assigned to each word) exploiting morphological/orthographic rules

text	stem	PoS	PoS features
Pavlov	PAVLOV	N	NOM SG PROPER
had	HAVE HAVE	V PCP2	PAST VFIN SVO SVO
shown	SHOW	PCP2	SVOO SVO SV
that	THAT THAT THAT THAT	ADV PRON DET CS	DEM SG CENTRAL DEM SG
salivation	SALIVATION	N	NOM SG

N: Noun
 V: Verb
 PCP2: Past Participle
 ADV: Adverb
 PRON: Pronoun
 DET: Determiner
 CS: Subordinating Conjunction

SVOO: Subject-Verb-Object-Object
 NOM: non-genitive
 VFIN: finite verb
 DEM: demonstrative

- Application of rules to select only one tag among those assigned to each word (rules exploit the word context)

Rules

- Rules are aimed to removing the cases that are not compatible with the context

- In ENGTWOL there are about 1100 rules

ADVERBIAL-THAT RULE

input: "that"

if

(+1 A/ADV/QUANT); // the following word is an adjective, adverb or quantifier
(+2 SENT-LIM); // and the following one is a sentence boundary
(NOT -1 SVOC/A); // and the preceding word is a verb like "consider"
// that admits an adjective as object (*I consider that good*)

then eliminate non-ADV tags

else eliminate ADV tag

- ENGTWOL has also probabilistic constraints and it may exploit syntactic information